

Install the GitHub MCP Server in VS Code

For this install I am using an older Ubuntu 24.04 and Node version 24.14.0

Before you begin this process, sign in to GitHub.com using one of the methods in VS Code. You must have `npm` installed in your Linux system before proceeding.

This is the first of three MCP Server installation on VS Code.

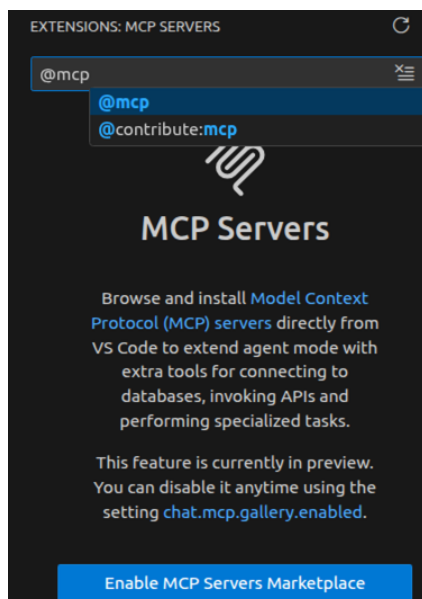
All three installations are being done in **June of 2026** and may have changed by the time the bootcamps are aired.

There will always be other ways to perform 99% of the tasks listed in this document, I had to choose a method and so below are the methods I chose.

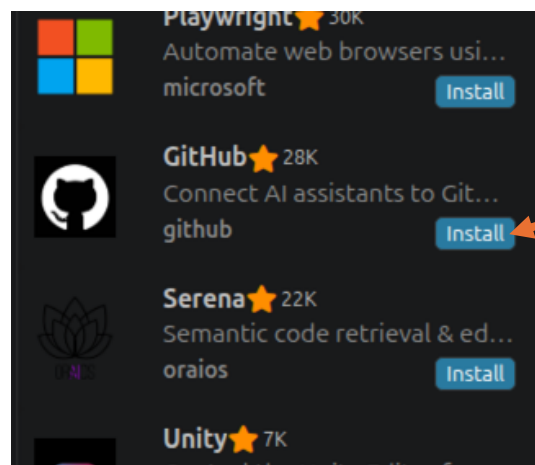
In VS Code, open any project and follow the instructions below. If you do not have an app, use the one in the downloaded files, it is the same zipped from file the GitHub part of the bootcamp. The zipped file is called `git-node-d2.zip`. Unzip that file and run `npm install`.

Part 1 – Environment Set Up

1. Open the Extensions panel (left of main editor window). You will see a search bar at the top of that window, begin typing `@mcp`. This will trigger the MCP list to show up.



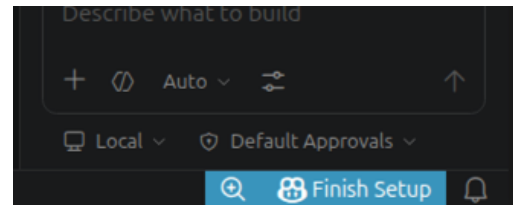
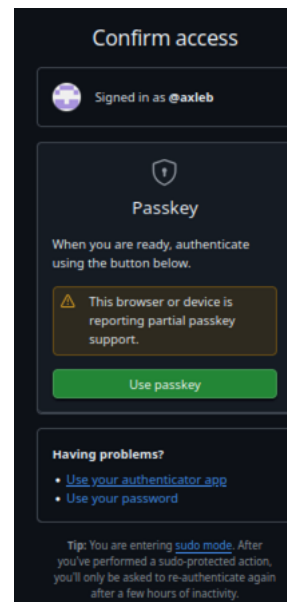
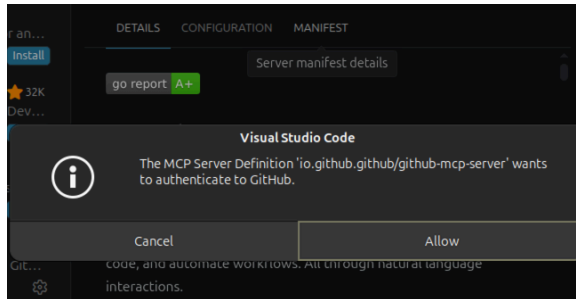
2. Choose GitHub and then click the blue Install button. You may be asked to sign in, if you have not already done so.



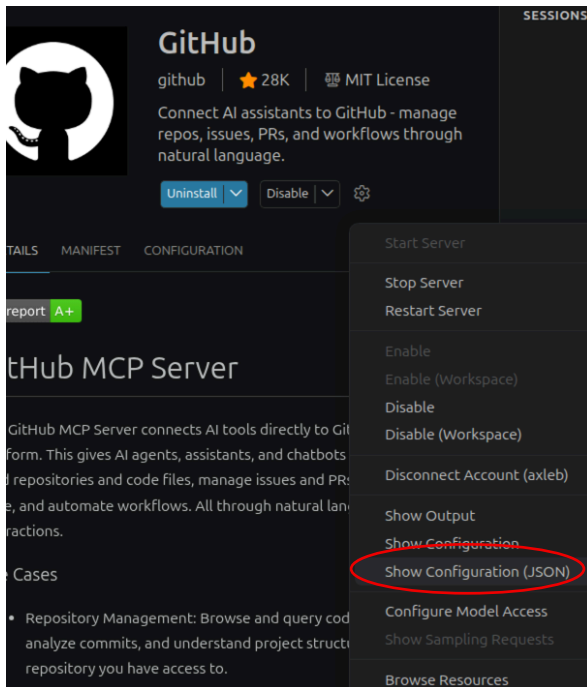
Click the Install button

Note: As of June, you will have to click the **Enable MCP Servers Marketplace** button.

The next three options will appear depending on your own setup. **This setup changes all the time**, I am just showing some of the configuration screens I have seen over the past few months.



3. If you have the extensions window still open, at the bottom right of the GitHub listing, there is a gear icon , click this to open the configuration file, so choose Show Configuration (JSON) . It's also available from the main editor window.



The image on the left is the main install configuration window. Clicking on the gear icon will reveal a sub menu where you can choose the option circled in red. That will bring up the file you see below:

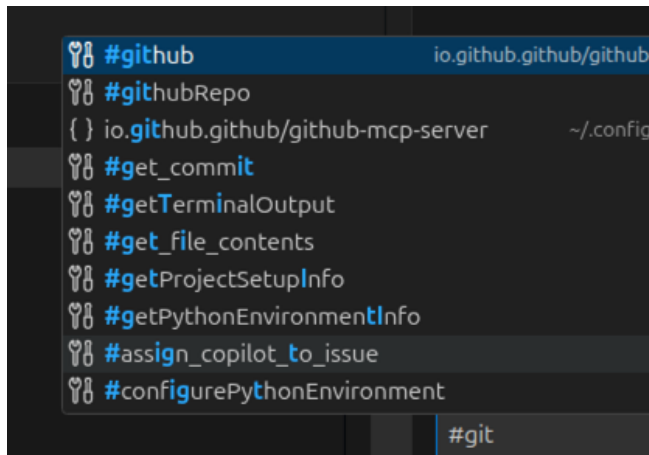
```
{
  "servers": {
    ✓ Running | Stop | Restart | 40 tools | 2 prompts | More...
    "io.github.github/github-mcp-server": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "gallery": "https://api.mcp.github.com",
      "version": "0.30.3"
    }
  },
  "inputs": [
```

4. At this point, we may have differing results. If your `mcp.json` file looks like this, then you are all set. Notice that the server is already running.

If you have the latest GitHub MCP server from early 2026, it now supports HTTP-based remote connections.

If not, you will have to do further configurations via Docker or npx (not covered in this bootcamp).

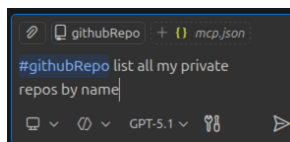
5. If you now go to your Chat input box in VS Code, you can access this server directly using the # tag:



In my VS Code Chat window, I started typing #git... and I can directly send a request via github or githubrepo.

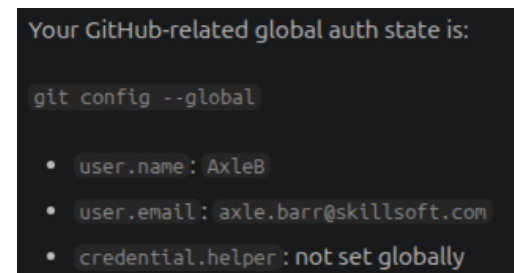
Once your LLM realizes that you are working with an MCP Server, it will then allow natural prompts. Adding the # is just for the first 2-3 prompts.

6. Based on #5, I can ask directly about my repos on GitHub.com:

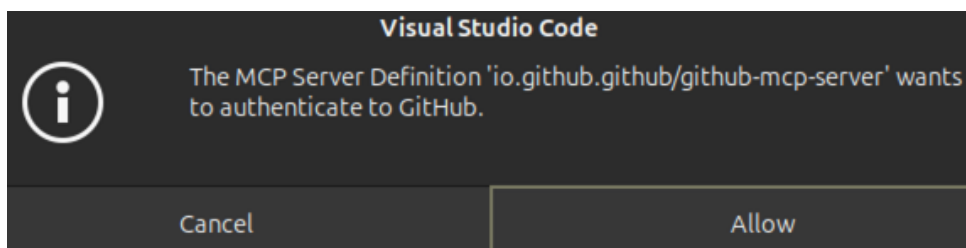


You must be logged in to GitHub Copilot at this point. The first time you work with the server it will take some time. Also, it may update.

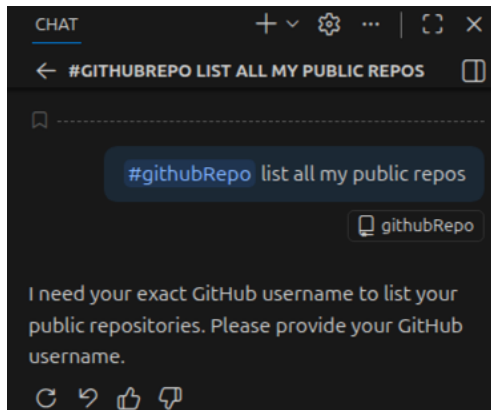
If you do not have any public or private repositories, you can ask for your Global Credentials like this: #github show me my global settings. It may ask for permission. Here is how it responded for me:



7. Depending on your configuration, you may be asked to authenticate for the first time:

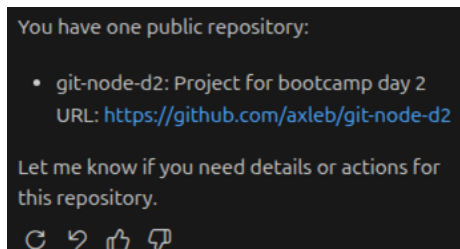


8. You may be asked for your credentials to get into GitHub from your Copilot Chat window:



Note, this part of the relationship between Git, GitHub, MCP Servers in VS Code is changing constantly. Your experience may be different depending on the time you participate in this bootcamp. You may or may not be asked for a token at this point.

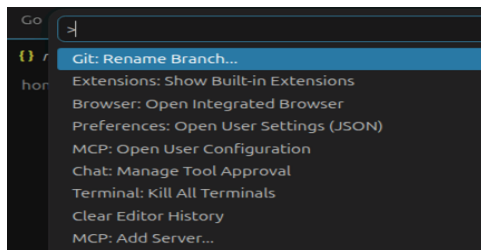
If you get to this point without much complications you will get an answer from Chat like this:



Once I signed in to GitHub Copilot, my VS Code Chat window automatically chose **GPT-5.3-Codex Medium** as the default LLM.

(Optional)

9. Lets do one more test to make sure that the GitHub MCP Server is in fact installed. You can do this for any MCP Server you install in the future. Open the command palette in VS Code, usually CTRL + SHIFT + P will work:



10. I asked the server to make one of my private repositories public, this is the result:

```
Run bash command?

gh repo edit axleb/
nodejs-intro-day01-25
--visibility private && gh
repo view axleb/
nodejs-intro-day01-25 --json
visibility,nameWithOwner

⚠️ Modifies repository visibility to private

Allow | ⌵ | Skip
```

Once I clicked on Allow, the transaction happened and my repo was public. I then did the opposite

11. My next prompt had to do with the files in the repo from above. The task was to find the name of the PDF document in that repo. It correctly returned Intro to NodeJS Day01.pdf.

12. Naturally you may want to find out the content of that file:

```
Here are the topics listed in that PDF:

1. Verify NodeJS
2. Building a Simple NodeJS App
3. Introduction to Node Packages and Express
4. Routing Basics
5. Installing Nodemon
6. Decomposing Routes
7. Decomposing Controllers
8. Appendix A: Simple HTTP Server
9. Appendix B: Linux Commands
10. Appendix C: Delete from Database
11. Appendix D: Robust App Improvements (End of Part 6)
12. Appendix E: NodeJS Advantages
```

Chat will keep asking you to allow the various terminal commands to run, you could of course set this to automatic.

Note, the above was for the GitHub MCP Server. For Git MCP Server check Appendix B.

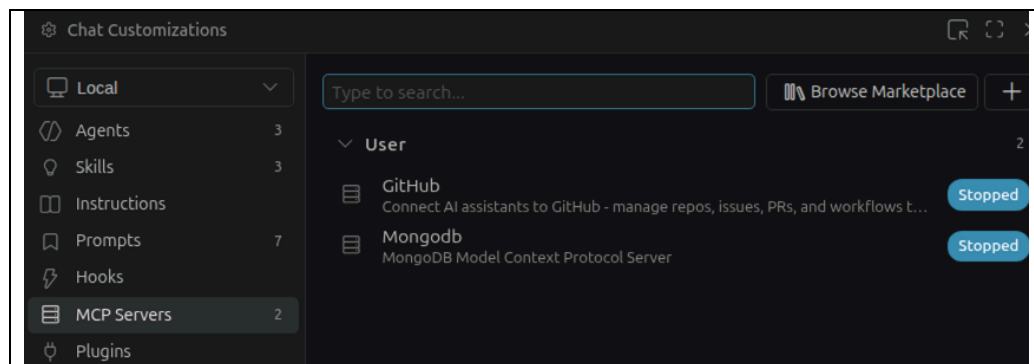
Part 2 – Working with the GitHub MCP Server

If you are continuing from the Git and GitHub bootcamps you will know that we created a project, sent it to GitHub and used a few Git and GitHub features. We can now do most of the tasks you learned on those two days directly from the Chat window.

Delete the remote repository now, if you still have it on GitHub.com. Do this directly on the site.

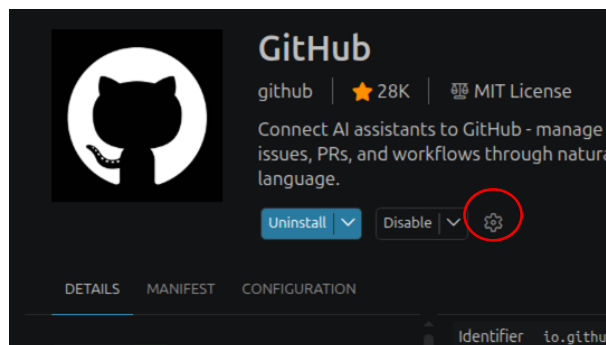
For the new participants joining just for today, there will be a zipped file that you can download and set up in your file system. This folder will already have a `.git` folder and a `.gitignore` file.

1. Start the server. If your GitHub MCP Server is NOT running, this is one way to start it up. Go to the top right corner of your VS Code IDE and click on the gear icon . Choose MCP Servers from the list on the left of the pop-up window:

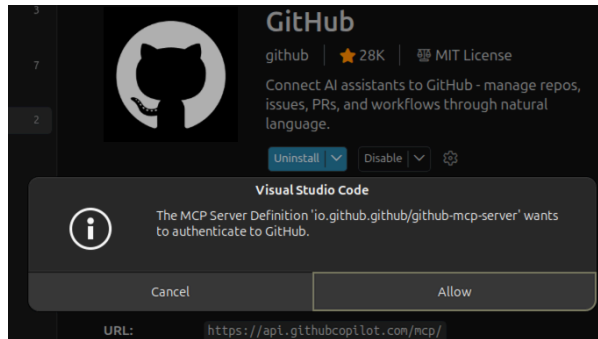


Click on the *Stopped* button to go to the MCP server settings for GitHub Server. Note, as of June, this has changed. Explanation during the bootcamp.

2. From #1 you can click on the gear icon to start the server:



3. You may be asked permissions to start the server:



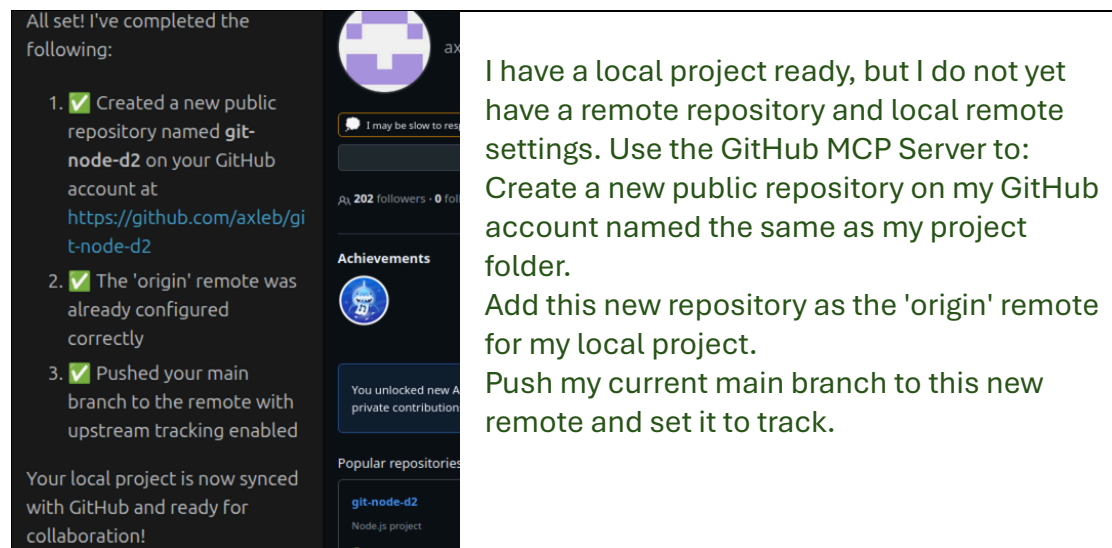
You may not see an indication that the server started but if you click the gear icon again, there will not be a Start option, if the server is already started.

4. We can now work freely with the Chat window to create our remote repository. Below is a possible prompt. Remember that this prompt may change based on the circumstances of the day of the bootcamp.

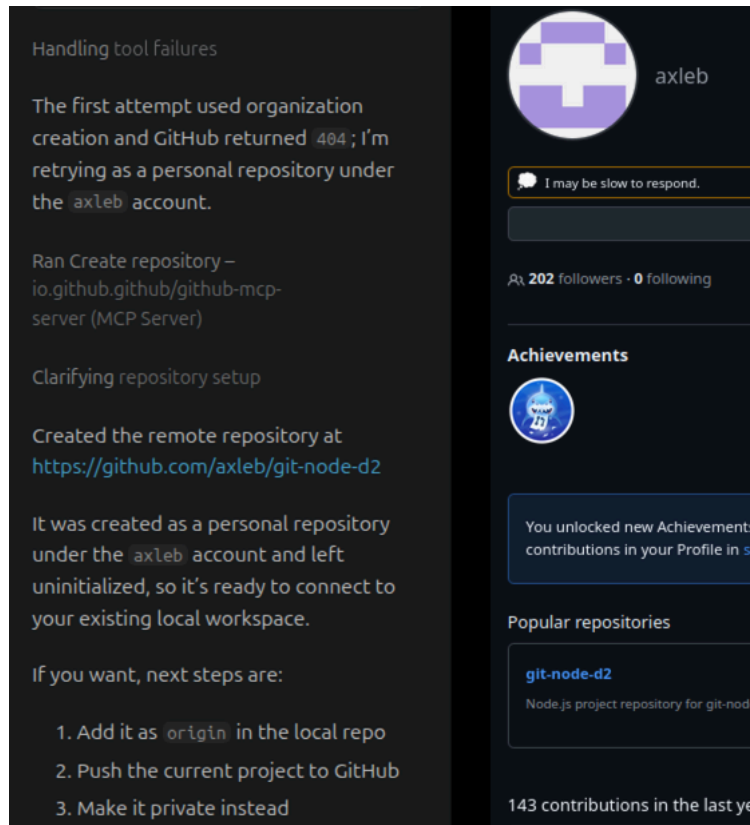
Once again I am use the GPT-5.3-Codex Medium Large Language Model.

The following steps, images, procedures may have changed by the time you are participating in this bootcamp.

Here is an image showing the response to my prompt and the browser on the right showing that the project was pushed correctly.



5. Below is my VS Code and Copilot Chat window on the left and my Mozilla Firefox browser on the right. The browser is currently at my repository github.com/axleb.



Notice that the repository has been created. It shows in blue in the image.

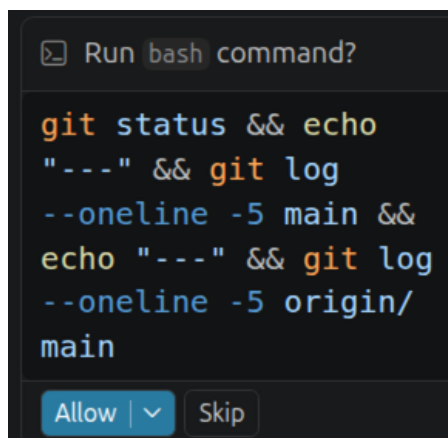
Because of the prompt that I used, this repository should be empty.

Notice also that Copilot is suggesting some steps below.

6. If #5 worked you can now ask for a reconciliation:

Check that the remote and local repositories are synchronized

After running the prompt above, I was asked to allow a bash command, see below:



Click *Allow*.

You could opt to break up the prompt in #4 into individual prompts.

Lets move on to installing the **MongoDB MPC Server**

Part 3 – Git Commands via Chat

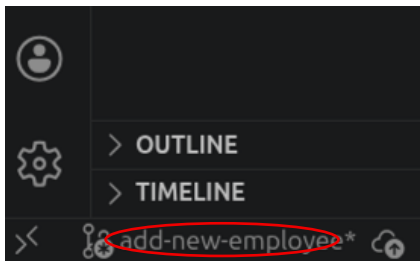
If you are working with the basic **NodeJS** starter project open the `controller.js` file. You will see that the `addemployee()` function is incomplete. We will now fix this function and use the Chat window to update our remote repo.

1. In the Chat window, prompt the tool to create a new feature branch called `add-new-employee`.

Create a new feature branch locally called `add-new-employee` and switch me to that branch

Here is the response. Now would be a good time to “Allow all commands in this session”. This option is available once you click the drop down arrow on the Allow button.

Verify that you are on the new branch by looking at the bottom left of VS Code:



2. Now that you are in the new branch open the `controllers.js` file. For the `addemployee()` function, add this code:

```
exports.addemployee=function(req, res){
  let empName = req.body.empName;
  let empPass = req.body.empPass;
  const Emp = new employee();
  Emp.empName = empName;
  Emp.empPass = empPass;
  Emp.save()
  .then(msg => {
    res.send({"message":"Created " + Emp.empName});
  })
  .catch(
    err => res.send({"message":err.message})
  );
};
```

Note, we cannot test this code as yet unless you already have your MongoDB database setup with a collection. We will test after we install the MongoDB MCP Server. For now just continue with Git commands.

3. Lets now get back to the main branch. In normal development you will test the changes first:

Switch to my main branch then merge the add-new-employee branch

At this point we should have two branches, main and add-new-employee. Just ask in the Chat window and you will get these two branches.

Here is the result:

Done.

- Switched to main successfully.
- Merged add-new-employee into main:
Git reports Already up to date, so there were no new commits to merge.
- Current status: main is aligned with origin/main, but you have unstaged local changes in:
 - `controller.js`
 - `package-lock.json`

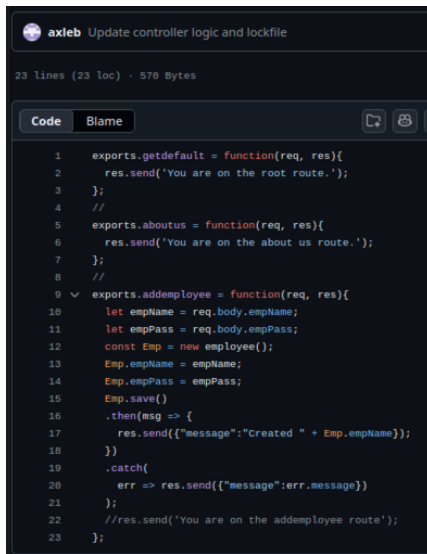
If you want, I can help you commit these local changes on main or move them to a feature branch first.

In the Chat window I will agree to what Chat is suggesting, commit the local changes on *main*.

4. The commit was done and it asked if I wanted to push to origin/main. I will prompt manually this time:

Yes push this commit to origin/main

You can now confirm on GitHub.com that the `controller.js` file was changed:

A screenshot of a GitHub commit interface. At the top, it says 'axleb Update controller logic and lockfile'. Below that, it shows '23 lines (23 loc) · 570 Bytes'. There are tabs for 'Code' and 'Blame'. The 'Code' tab is selected, showing a diff of the `controller.js` file. The code includes functions for `getdefault`, `aboutus`, and `addemployee`. The `addemployee` function is highlighted with a blue selection bar.

```
1 exports.getdefault = function(req, res){
2   res.send('You are on the root route.');
```

```
3 };
4 //
5 exports.aboutus = function(req, res){
6   res.send('You are on the about us route.');
```

```
7 };
8 //
9 exports.addemployee = function(req, res){
10   let empName = req.body.empName;
11   let empPass = req.body.empPass;
12   const Emp = new employee();
13   Emp.empName = empName;
14   Emp.empPass = empPass;
15   Emp.save()
16   .then(msg => {
17     res.send({"message": "Created " + Emp.empName});
18   })
19   .catch(
20     err => res.send({"message": err.message})
21   );
22   //res.send('You are on the addemployee route');
```

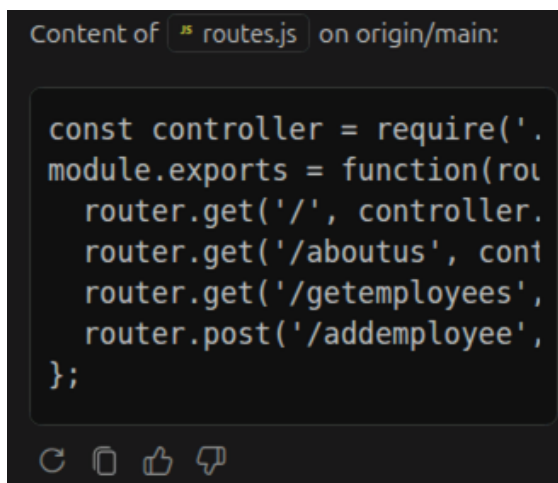
```
23 };
```

Note, if Copilot does not ask you directly, you can pass a prompt to do the push for you.

A few other tasks you can try:

- Open a new issue or comment on an existing issue.
- Work with comments and conversations
- Search for files or code strings

5. I made a change directly on GitHub. The `routes.js` file was edited remotely and then I asked Chat to read the entire file for me. This is what it showed:

A screenshot of a ChatGPT interface. At the top, it says 'Content of routes.js on origin/main:'. Below that, it shows the content of the `routes.js` file. The code includes `const controller = require('.')` and `module.exports = function(router) {` followed by several `router.get` and `router.post` calls. The `router.get('/getemployees', controller.getEmployees);` line is highlighted with a blue selection bar.

```
Content of routes.js on origin/main:

const controller = require('.');
module.exports = function(router) {
  router.get('/', controller.get);
  router.get('/aboutus', controller.getAboutus);
  router.get('/getemployees', controller.getEmployees);
  router.post('/addemployee', controller.addEmployee);
};
```

I added the `/getemployees` line.

6. Now we can do a sync by fetching and merging, but with MCP Server and Chat we don't need to worry about the technical jargon. Here is my prompt:

```
Check if my local repository is in sync with the remote repository. Fetch the latest changes from origin, compare the commit hashes of my local branch against its remote branch, and list any commits I am missing or ahead by.
```

Here is the result:

Fetch and comparison complete.

- Local branch hash:
190e63cb7ac998684c114835642cf88f7db20ed9
- Remote branch hash (origin/main):
5512dd106a3349bc4dee49082ed698b120359c22
- Ahead/behind result: you are behind by 1 commit, ahead by 0 commits

Commits you are missing from local (present on remote):

- 5512dd1 Add route for getting employees

Commits you are ahead by (present locally but not on remote):

- none

Your local repository is not fully in sync; you need to pull 1 remote commit.

Appendix A – Further Configurations for JSON Config File

Here are common configurations for the mcp.json file. For example, if you need to connect to a remote GitHub MCP server but require a personal access token (PAT), hardcoding secrets into your .vscode/mcp.json is a security risk.

This configuration defines an input prompt so VS Code asks you for the token securely and masks your typing.

```
        "url": "https://api.githubcopilot.com/mcp/",
        "gallery": "https://api.mcp.github.com",
        "version": "1.6.0",
        "headers": {
            "Authorization": "Bearer ${input:github_mcp_pat}"
        },
    },
    "inputs": [
        {
            "type": "promptString",
            "id": "github_mcp_pat",
            "description": "GitHub Personal Access Token",
            "password": true
        }
    ]
}
```

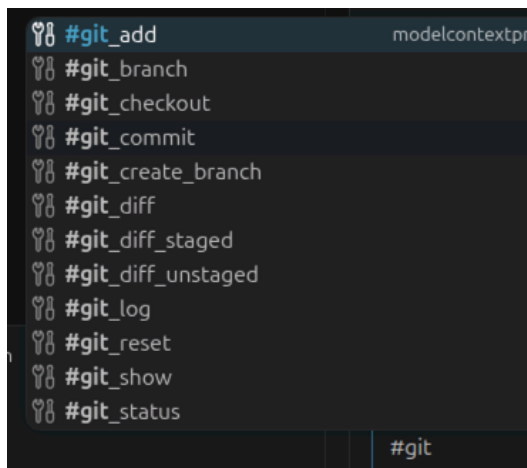
Appendix B – Git MCP Server

Although you can do a lot in VS Code for Git based work, using the official MCP Server makes this task just a little bit easier. I did not find an official Git MCP Server in the MCP marketplace, so here is the direct configuration (Git is not an extension).

```
        "gallery": "https://api.mcp.github.com",
        "version": "1.6.0"
    },
    "modelcontextprotocol/server-git": {
        "type": "stdio",
        "command": "uvx",
        "args": [
            "mcp-server-git"
        ]
    },
    "inputs": []
```

You can also install this server via the Command Palette and typing MCP: Add Remote Server. There may be other ways.

Once installed, if you go into the Chat window and type #git, this is what shows up as of June 2026:



But you can also ask directly in Chat:

"Show me the current status of my local repository. What files have changed?"

After putting that prompt into Chat, this is what it reported:

Checking your repo state now and I'll report exactly which files are changed and whether they're staged or unstaged.

I'm running a direct git status check in your workspace repo so I can list changed files with their exact staging state.

Ran `git_status` - modelcontextprotocol/server-git (MCP Server)

Current local repo status:

- Branch: `main` (up to date with `origin/main`)
- Changed files (unstaged):
 - `routes.js`
- Staged changes: none
- Untracked files: none

I then asked it to show me the last five commits:

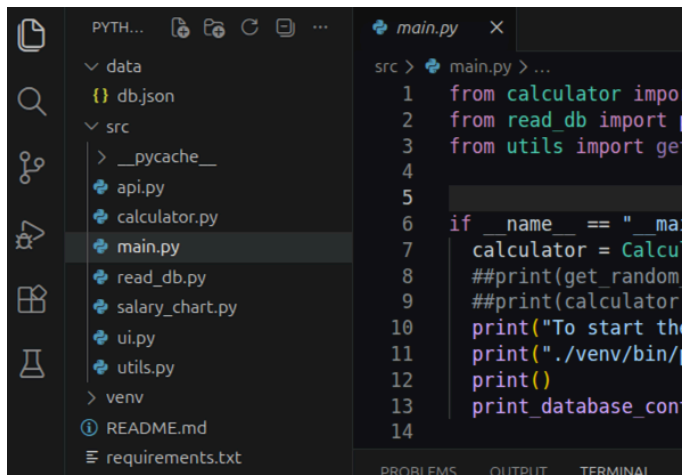
Last 5 commits on this branch (newest to oldest), with what changed:

1. 2c97870 - Updated employee model
Changed `employee.js` : fixed the MongoDB connection string typo by closing the URI quote after Employees.
2. 7a7a761 - Add files via upload
Changed `employee.js` : added the Mongoose setup, database connect call, employee schema (`empName`, `empPass`, `created`), and model export.
3. c9bbd18 - Create employee.js
Added `employee.js` : initial file creation (empty placeholder at that point).
4. 816785c - Merge pull request #1 from

You can also ask it to create branches or stage the changes. Try doing a deep code review or advanced tasks like git stash.

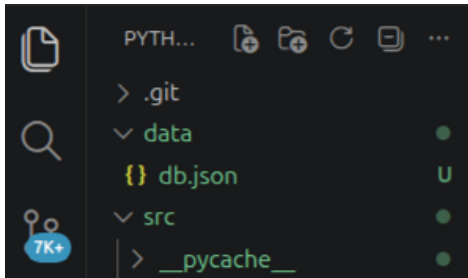
There is a `mcp.json` file already configured with Git and GitHub in the Assets folder if you wish to use it.

Appendix C – Turn any Folder into a Git Tracked Folder



Here I have a Python application in a folder called python-boilerplate. I am working with VS Code for this app. Here are the steps to convert to a Git repository that can later be pushed to GitHub or any online repository service:

1. The first command is just `git init`. That command will insert the hidden `.git` directory.



2. (Optional) This step is for Python developers only. You can go to this location and download the `.gitignore` file.

<https://github.com/github/gitignore/blob/main/Python.gitignore>

This file will direct Git to not upload certain files and folders that will be recreated anyway once the project is downloaded. Upload it to the project folder.

3. The next command will add all the files/folders except any listed in the `.gitignore` file to the repository: `git add .`

4. This is the status after the first three commands for this Python project:

```
▼ TERMINAL
● axle@pc0480:~/Documents/python-boilerplate$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   README.md
    new file:   data/db.json
    new file:   requirements.txt
    new file:   src/api.py
    new file:   src/calculator.py
    new file:   src/main.py
    new file:   src/read_db.py
    new file:   src/salary_chart.py
    new file:   src/ui.py
    new file:   src/utils.py
```

5. At this point you can perform your first commit: `git commit -m "First Commit of my Phthon Boilerplate project"`.
6. After running #5 command:

```
● axle@pc0480:~/Documents/python-boilerplate$ git status
On branch master
nothing to commit, working tree clean
○ axle@pc0480:~/Documents/python-boilerplate$
```

7. Now your Python boilerplate project is ready for GitHub interactions.